

DSPO: STABLE AND EFFICIENT POLICY OPTIMIZATION FOR AGENTIC SEARCH AND REASONING

Anonymous authors
Anonymous submission

ABSTRACT

Enhancing LLMs with the ability to actively search external knowledge is crucial for complex and real-world tasks. Current approaches either rely on prompting to elicit the model’s innate agent capabilities, or suffer from performance ceilings and collapse when applying RL to complex interactive tasks, leaving their true agentic potential untapped. To address this, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, an improved RL algorithm designed for robust agent training through sequence-level optimization and dynamic sample filtering. We train our model purely through RL to interleave multi-turn search and reasoning, obviating the need for supervised demonstration data. The gains are large and consistent. Evaluated on seven QA benchmarks under an identical BM25 setup, our 7B model raises the average substring-EM score to **0.531**, a **34.1% relative** improvement over the comparable 7B Search-R1 (GRPO) baseline (0.377). Strikingly, this 7B model matches or exceeds the much larger 14B Search-R1 models on average, and on the demanding multi-hop HotpotQA benchmark it surpasses the 14B GRPO model by nearly **9% relative** (**0.613** vs. 0.563). These improvements are delivered while maintaining a smooth, monotonic training trajectory, in contrast to the catastrophic policy collapse observed for token-level baselines.

1 INTRODUCTION

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023; Zhao et al., 2023) have demonstrated exceptional performance across a spectrum of specialized tasks, including math (Shao et al., 2024; Trinh et al., 2024; Yu et al., 2025), coding (Zheng et al., 2023; Yang et al., 2024), and creative writing (Chakrabarty et al., 2024; Marco et al., 2024). However, a fundamental limitation persists: their knowledge is inherently static, confined to the data on which they were trained. To overcome this knowledge cutoff, a dominant approach is to equip LLMs with search capabilities, transforming them into agents that can actively query external knowledge sources (Jin et al., 2025b). This ability is a prime example of tool-calling (Schick et al., 2023), where the model learns to interact with an external search tool to solve problems it cannot answer alone. Mastering this skill requires learning a complex, multi-step policy, framing the task as a sequential decision-making problem ideal for Reinforcement Learning (RL).

Unlike Supervised Fine-Tuning (SFT), which relies on costly static demonstrations and fails to teach exploration, RL provides a framework for LLMs to learn effective policies through trial-and-error (Chu et al., 2025). Consequently, value-free methods like Group Relative Policy Optimization (GRPO) (Shao et al., 2024) have become a dominant paradigm, prized for their simplicity and reduced memory overhead. However, despite its success in more constrained tasks, applying GRPO to the open-ended domain of interactive search reveals critical instabilities (Jin et al., 2025b; Yu et al., 2025; Cui et al., 2025; Liu et al., 2025). This fragility stems from two fundamental flaws. First, as identified by Zheng et al. (2025), GRPO’s token-level objective is ill-posed when paired with a sequence-level reward, creating high-variance gradients that destabilize training. Second, the sparse rewards inherent to search tasks often yield sample groups with homogeneous outcomes (e.g., all successes or all failures), causing the advantage signal to collapse and providing no learning signal, which severely hinders sample efficiency (Yu et al., 2025; Liu et al., 2025).

To address these core challenges of instability and inefficient learning, we introduce **Dynamic-filter Sequence-level Policy Optimization (DSPO)**. Our algorithm synthesizes and refines key principles from recent policy optimization research. DSPO adopts the sequence-level optimization from GSPO (Zheng et al., 2025) to match the unit of optimization with the unit of reward. This aligns the optimization objective with the reward signal, fundamentally stabilizing the learning process for long-horizon reasoning tasks. Furthermore, DSPO incorporates a dynamic outcome-based filtering mechanism inspired by DAPO (Yu et al., 2025). This component actively constructs training batches from rollout groups containing both successful and unsuccessful outcomes for each prompt. It guarantees the advantage signal $\hat{A}_i$ to be effective and stable. By integrating these two components into a single, coherent framework, DSPO

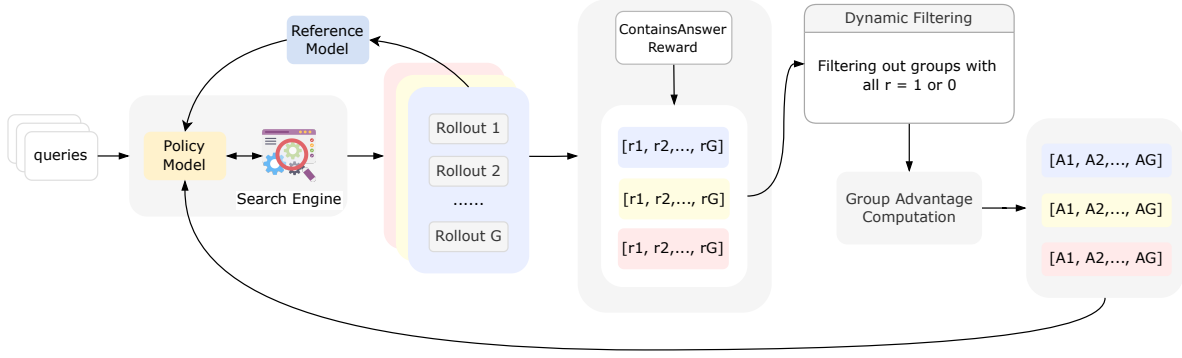


Figure 1: An overview of the DSPO training loop. For a given query, the policy model generates a group of G trajectories by interacting with the search environment, and each trajectory is assigned a sparse terminal reward. The two mechanisms that drive our empirical gains are highlighted: the **dynamic filter** discards groups with homogeneous outcomes and keeps sampling until a batch is filled, guaranteeing that every training batch carries a non-zero, effective advantage signal, while advantages are then aggregated and used to update the policy through a stabilizing **sequence-level objective**.

provides a stable and high-performance algorithm designed for complex, multi-turn search and reasoning tasks. The payoff is quantitatively decisive: under matched training and evaluation conditions, our 7B model improves the seven-benchmark average from 0.377 to **0.531**, a **34.1% relative improvement** over the same-sized Search-R1 (GRPO, 7B) baseline (Jin et al., 2025b). The advantage does not stop there. On complex multi-hop benchmarks such as HotpotQA, our 7B model even surpasses the twice-as-large 14B Search-R1 counterpart (Jin et al., 2025a), outperforming it by **nearly 9% relative (0.613 vs. 0.563)**—evidence that the gains come from the algorithm rather than from model scale.

In summary, our main contributions are as follows:

- We propose DSPO, an improved RL algorithm that overcomes the core instability and sample-inefficiency issues in training agentic search models. It achieves this by unifying two key principles into a single cohesive framework: **sequence-level optimization** for robust policy updates and **dynamic outcome-based filtering** for a dense and effective learning signal.
- We demonstrate DSPO’s substantial performance gains through rigorous, condition-matched benchmarking. Averaged over seven QA benchmarks, our 7B model reaches **0.531** substring-EM, a **34.1% relative improvement** over the comparable 7B baseline (0.377). More strikingly, this 7B model equals or beats the 14B Search-R1 models on average and, on complex multi-hop QA, outperforms the 14B GRPO counterpart on HotpotQA by nearly **9% relative (0.613 vs. 0.563)**. The advantage is consistent, holding on the majority of individual benchmarks rather than resting on a single dataset.
- We provide extensive empirical evidence for DSPO’s superior training stability, showing it converges along a smooth, monotonic reward trajectory to the highest reward level while token-level baselines collapse. Crucially, every reported number is obtained with only a basic BM25 retriever, isolating the measured gains to the robustness of our algorithm rather than to a stronger retrieval pipeline.

2 RELATED WORK

2.1 RL FOR LLMs

The landscape of RL for LLMs has evolved rapidly, moving from foundational Reinforcement Learning from Human Feedback (RLHF) methods that use PPO and explicit reward models (Ouyang et al., 2022; Christiano et al., 2017; Schulman et al., 2017) to simpler, direct-optimization frameworks like DPO (Rafailov et al., 2023). A key shift towards value-free optimization is marked by Group Relative Policy Optimization (GRPO) (Shao et al., 2024), which simplifies training by deriving a reward signal from group statistics. However, GRPO’s token-level objective is known to cause training instability (Liu et al., 2025; Cui et al., 2025), prompting several targeted improvements. GSPO addresses this by shifting to a sequence-level objective to match the unit of reward (Zheng et al., 2025), while DAPO tackles inefficient learning from sparse rewards with a dynamic outcome-based sampling mechanism (Yu et al., 2025).

In a similar vein, GMPO stabilizes the token-level objective using a geometric-mean aggregation to reduce sensitivity to outliers (Zhao et al., 2025). Despite these advances, our own controlled comparisons show that, taken individually, these algorithms still fall short on interactive search: token-level variants collapse during training, and the sequence-level objective alone plateaus at a suboptimal reward ceiling (quantified in Table 2 and Figure 3). Building on this research, we propose an improved algorithm that unifies sequence-level optimization with dynamic outcome-based filtering into a single framework, and we show it closes exactly these two failure modes on autonomous search agents.

2.2 LLMs WITH AGENTIC RETRIEVAL

To mitigate the static knowledge limitations of LLMs, RAG integrates external retrievers to dynamically incorporate evolving information (Lewis et al., 2020; Gao et al., 2023). Classic RAG frameworks employ dense retrievers to fetch relevant documents, which are then concatenated into the LLM’s input for generation (Karpukhin et al., 2020). However, these approaches often rely on fixed pipelines, limiting autonomy in complex, multi-turn scenarios. Recently, research has evolved toward agentic paradigms, where LLMs act as autonomous agents capable of planning, searching, and reasoning iteratively. Frameworks like ReAct synergize reasoning and acting, enabling LLMs to interact with tools for tasks such as web navigation (Yao et al., 2023), while multi-agent systems, including AutoGen, facilitate collaborative workflows (Wu et al., 2024). Recent innovations emphasize agentic RAG and RL integration, where agents enhance retrieval through decision-making. Wu et al. (2025) introduce Agentic Reasoning, a framework integrating external tools for streamlined LLM reasoning. Some RL-integrated approaches (Jin et al., 2025b; Chen et al., 2025; Song et al., 2025) train LLMs to interleave reasoning and search using purely RL. The end-to-end paradigm internalizes agent capabilities and can avoid the engineering overhead of multi-agent frameworks. However, these methods still grapple with training instability and a performance ceiling in the open-ended search domain, both of which we measure directly against DSPO in our experiments. Our work confronts these bottlenecks head-on: DSPO provides a robust and efficient training framework that yields stable policy optimization and, as the seven-benchmark comparison shows, translates that stability into a 34.1% relative average gain over the strongest same-sized RL baseline.

3 METHODOLOGY

In this section, we first formulate the task of agentic search as an RL problem and review prior policy optimization algorithms, pinpointing the specific limitations that our later experiments quantify. We then introduce our proposed algorithm, **Dynamic-filter Sequence-level Policy Optimization (DSPO)**, detailing the two components responsible for its measured gains in training stability and sample efficiency. Finally, we present the integrated training algorithm.

3.1 PRELIMINARIES

Policy Gradient Methods for LLMs. Training LLMs via RL often employs policy gradient methods like PPO (Schulman et al., 2017), a popular algorithm for LLM alignment. It optimizes a policy π_θ by maximizing a clipped surrogate objective function using samples from an old policy $\pi_{\theta_{\text{old}}}$. The objective, averaged over tokens, is given by:

$$J_{\text{PPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{|y|} \sum_{t=1}^{|y|} \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (1)$$

where $r_t(\theta) = \frac{\pi_\theta(y_t|x, y_{<t})}{\pi_{\theta_{\text{old}}}(y_t|x, y_{<t})}$ is the token-level importance ratio. However, PPO relies on a separately trained value model to estimate token-level advantages $\hat{A}_t$ via Generalized Advantage Estimation (GAE) (Schulman et al., 2015), introducing significant memory overhead and can be a source of instability.

To address this, GRPO (Shao et al., 2024) was proposed. GRPO eliminates the need for a value model by sampling a group of G responses $\{y_i\}_{i=1}^G$ for a given prompt x . It then calculates the advantage of each response by normalizing its reward against the group’s statistics. Like PPO, it optimizes the objective at the token level:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot|x)} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \min \left(r_{i,t}(\theta) \hat{A}_i, \text{clip}(r_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) - \beta D_{KL}(\pi_\theta || \pi_{\text{ref}}) \right], \quad (2)$$

where $r_{i,t}(\theta) = \frac{\pi_\theta(y_{i,t}|x,y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x,y_{i,<t})}$, and the advantage for every token $y_{i,t}$ in a response y_i is set to the same sequence-level value:

$$\hat{A}_{i,t} = \hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R)}, \quad (3)$$

Crucially, all tokens within a given response y_i share the same advantage $\hat{A}_i$, which is derived from the sequence-level reward.

Agentic Search as a Markov Decision Process. We model the iterative process of agentic search and reasoning as a sequential decision-making problem, formalized as a discrete-time, finite-horizon Markov Decision Process (MDP).

- **State (s_t):** At turn t , the state s_t encodes the entire interaction history: the initial question q , all previously generated thoughts and search queries, and the retrieved evidence returned by the environment. Because search results must be interpreted and integrated into subsequent decisions, the state necessarily grows with the trajectory, creating long-horizon dependencies that conventional token-level RL struggles to optimize.
- **Action (a_t):** An action is a full textual segment generated by the policy π_θ , consisting of free-form reasoning followed by a decision. The action terminates either with a `</tool_call>` token, which triggers a search, putting the results within `</tool_response>`, or with a `</answer>` token, which ends the trajectory. This structured action space forces the agent to learn not only *what* to generate but also *when* to search—an aspect that introduces significant variability in trajectory length.
- **Policy (π_θ):** The policy π_θ is the underlying LLM, generating tokens autoregressively conditioned on the state. The policy conditions on the full history, but the optimization target is calculated only on the model-generated thoughts and actions, masking out the retrieved content from the environment (Jin et al., 2025b). Optimizing this policy requires credit assignment over long sequences in which many intermediate reasoning steps do not receive direct supervision or reward, further emphasizing the need for stable sequence-level optimization.
- **Trajectory (τ):** A trajectory $\tau = (s_1, a_1, \dots, s_T, a_T)$ records all reasoning and tool interactions taken for a given question, including both model-generated actions and environment-returned search results. Because the final reward is assigned at the level of the entire trajectory, the optimization problem is fundamentally sequence-level: every early decision can influence the eventual answer correctness.
- **Reward ($R(\tau)$):** We employ a sparse terminal reward: a trajectory receives $R = 1$ if the final answer contains the ground-truth text, and $R = 0$ otherwise:

$$R(\tau) = \begin{cases} 1 & \text{if } a_{\text{gold}} \subseteq a_{\text{pred}}, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

RL with a Search Engine. Following Jin et al. (2025b), we explicitly model the search engine, denoted as $\mathcal{S}$, as part of the environment. The policy LLM π_θ learns to generate trajectories by interleaving reasoning with calls to $\mathcal{S}$. The overall optimization problem is to find a policy that maximizes the expected reward, regularized by a KL divergence term to prevent large deviations from a reference policy π_{ref} :

$$\max_{\pi_\theta} \mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta(\cdot|x;\mathcal{S})} [R(x, y)] - \beta D_{\text{KL}} [\pi_\theta(y|x;\mathcal{S}) || \pi_{\text{ref}}(y|x;\mathcal{S})]. \quad (5)$$

Here, $y \sim \pi_\theta(\cdot|x;\mathcal{S})$ signifies that the trajectory y is generated through a multi-step process involving both the policy’s token generation and the information returned by the search engine $\mathcal{S}$.

Motivation for DSPO. While frameworks like Search-R1 (Jin et al., 2025b) have successfully framed agentic search as an RL problem, applying conventional algorithms like PPO or GRPO faces significant hurdles that our experiments later quantify as concrete reward-curve collapse and lost accuracy. The open-ended nature of the search environment exacerbates the instability of token-level optimization. A core issue is the fundamental mismatch between the unit of sequence-level reward assignment and the unit of token-level optimization (Zheng et al., 2025). This discrepancy leads to high-variance gradient estimates that accumulate over long trajectories, often culminating in policy collapse. Furthermore, the sparse binary reward signal means many training batches may contain only successful or only unsuccessful trajectories, yielding an abnormal advantage and thus no learning signal, which drastically reduces sample efficiency (Yu et al., 2025; Liu et al., 2025). DSPO targets these two failure modes precisely, and Section 4 isolates the contribution of each through a controlled ablation.

3.2 DYNAMIC-FILTER SEQUENCE-LEVEL POLICY OPTIMIZATION

DSPO introduces two key innovations over prior methods: (1) it performs policy optimization at the sequence level, aligning the training objective with the trajectory-based reward structure, and (2) it incorporates a dynamic filtering mechanism to ensure every training batch provides a high-quality, non-zero learning signal. The entire training process, which integrates these components, is depicted in Figure 1.

3.2.1 SEQUENCE-LEVEL POLICY OPTIMIZATION FOR ENHANCED STABILITY

Inspired by GSPO (Zheng et al., 2025), we replace the unstable token-level importance ratio with a theoretically grounded sequence-level counterpart. The sequence-level importance ratio $s_i(\theta)$ for a response y_i is defined as the geometric mean of its token-level ratios:

$$s_i(\theta) = \left(\frac{\pi_\theta(y_i|x)}{\pi_{\theta_{\text{old}}}(y_i|x)} \right)^{\frac{1}{|y_i|}} = \exp \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \log \frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\theta_{\text{old}}}(y_{i,t}|x, y_{i,<t})} \right). \quad (6)$$

This length normalization is crucial for reducing variance and ensuring that $s_i(\theta)$ remains within a consistent numerical range regardless of sequence length, which is vital for stable clipping.

Gradient Analysis. The gradient analysis below shows why DSPO enhances the stability. The gradient of the token-level GRPO objective (unclipped) scales each token’s log-probability gradient by a noisy, token-specific weight $r_{i,t}(\theta)$. In contrast, the gradient of our sequence-level objective scales the average log-probability gradient of the entire sequence by a single, more stable sequence-level weight $s_i(\theta)$:

$$\nabla_\theta J_{\text{GRPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot \sum_{t=1}^{|y_i|} r_{i,t}(\theta) \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (7)$$

$$\nabla_\theta J_{\text{DSPO}} \propto \mathbb{E} \left[\hat{A}_i \cdot s_i(\theta) \cdot \sum_{t=1}^{|y_i|} \nabla_\theta \log \pi_\theta(y_{i,t} | \dots) \right] \quad (8)$$

By applying a single, holistic correction factor to the entire trajectory, DSPO avoids the accumulation of token-level noise that plagues prior methods, leading to fundamentally more stable training—an effect visible in Figure 3, where DSPO rises monotonically while its token-level counterparts collapse. In parallel, the dynamic filtering mechanism guarantees a well-defined advantage signal $\hat{A}_i$ by constructing training batches from rollout groups that contain both successes and failures, thus preventing wasted samples in sparse-reward environments. The ablation in Table 2 confirms that both effects are necessary: removing either component sharply lowers the average score.

3.2.2 DYNAMIC OUTCOME-BASED FILTERING FOR EFFICIENT LEARNING

The sparse binary nature of our reward function poses a challenge for group-based advantage estimation. If all G responses in a group are correct ($R = 1$) or all are incorrect ($R = 0$), the normalized advantage $\hat{A}_i$ becomes zero or undefined. Such batches do not provide a useful gradient signal, wasting computational resources.

To overcome this, DSPO incorporates a dynamic filtering mechanism inspired by DAPO (Yu et al., 2025). During sampling, we only retain groups of trajectories that contain a mix of successful and unsuccessful outcomes. A group $\{y_i\}_{i=1}^G$ is used for training only if its rewards $\{R_i\}_{i=1}^G$ satisfy:

$$0 < \sum_{i=1}^G R_i < G. \quad (9)$$

This ensures that the reward variance within every training group is non-zero, guaranteeing a meaningful advantage signal. This dynamic selection curates a high-quality dataset for each policy update, transforming a sparse reward problem into a dense and efficient learning signal. Its practical impact is large: as reported in Table 2, adding this single mechanism lifts the seven-benchmark average from 0.313 to 0.531.

Algorithm 1 Dynamic-filter Sequence-level Policy Optimization (DSPO)

```
1: Input: Initial policy  $\pi_{\theta_0}$ , fixed reference policy  $\pi_{\text{ref}}$ , prompt dataset  $\mathcal{D}$ , group size  $G$ , batch size  $B$ , search tool  $\mathcal{R}$ .
2: Initialize policy  $\pi_{\theta} \leftarrow \pi_{\theta_0}$ .
3: for each training step do
4:    $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ .
5:   Initialize training buffer  $\mathcal{B} \leftarrow \emptyset$ .
6:   while  $|\mathcal{B}| < B$  do
7:     Sample a prompt  $x \sim \mathcal{D}$ .
8:     Generate a group of  $G$  trajectories  $\{y_i\}_{i=1}^G$  using  $\pi_{\theta_{\text{old}}}$  and the search tool  $\mathcal{R}$ .
9:     Compute terminal rewards  $\{R_i\}_{i=1}^G = \{\text{ContainsAnswer}(y_i, y_{\text{gold}})\}_{i=1}^G$ .
10:    if  $0 < \sum_{i=1}^G R_i < G$  then ▷ Dynamic outcome-based filtering
11:      Add  $(x, \{y_i\}_{i=1}^G, \{R_i\}_{i=1}^G)$  to  $\mathcal{B}$ .
12:    end if
13:  end while
14:  for each  $(x, \{y_i\}, \{R_i\})$  in  $\mathcal{B}$  do
15:    Compute advantages  $\{\hat{A}_i\}_{i=1}^G$  via group normalization of  $\{R_i\}$ .
16:    Compute sequence-level importance ratios  $\{s_i(\theta)\}_{i=1}^G$  using Eq. 6.
17:    Compute the DSPO loss for the group using Eq. 11, applying masks to retrieved tokens.
18:  end for
19:  Update policy parameters  $\theta$  by taking a gradient step on the total loss from  $\mathcal{B}$ .
20: end for
```

3.3 THE DSPO OBJECTIVE AND TRAINING ALGORITHM

By integrating these components, we arrive at the final DSPO objective. For each valid group from the filtered sample space $\mathcal{D}_{\text{filtered}}$, we compute the advantage $\hat{A}_i$ using group-relative normalization:

$$\hat{A}_i = \frac{R_i - \text{mean}(R)}{\text{std}(R) + \delta}, \quad (10)$$

where δ is a small constant for numerical stability. The policy π_{θ} is updated by maximizing:

$$J_{\text{DSPO}}(\theta) = \mathbb{E}_{(x, \{y_i\}) \in \mathcal{D}_{\text{filtered}}} \left[\frac{1}{G} \sum_{i=1}^G \min \left(s_i(\theta) \hat{A}_i, \text{clip}(s_i(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i \right) \right], \quad (11)$$

where s_i is the sequence-level importance ratio defined as the geometric mean of the token-level ratios. Crucially, during likelihood calculation, we apply loss masking to all tokens retrieved from the search tool following Jin et al. (2025b). This ensures the model learns to utilize external knowledge for reasoning, not simply to reproduce it. The full training process is detailed in Algorithm 1.

4 EXPERIMENTS

In this section, we empirically validate the effectiveness of DSPO through a set of condition-matched comparisons. Our experiments are organized around three quantitative questions: (1) does DSPO outperform strong external baselines on challenging question-answering benchmarks, and by how much; (2) does it exhibit measurably greater training stability, avoiding the catastrophic collapse seen in token-level baselines; and (3) do both of its core components contribute to the observed gains, as isolated by a controlled ablation? As we show, DSPO answers all three affirmatively, with a 34.1% relative average improvement over the same-sized baseline, a smooth non-collapsing reward curve, and a clear per-component decomposition of that gain.

4.1 EXPERIMENTAL SETUP

Following Search-R1 (Jin et al., 2025b), and as shown in Table 1, we adopt the same prompt template to instruct the model’s actions during the search task, including `<think>`, `<tool_call>` and `<answer>`. Reusing this template verbatim keeps our setup directly comparable to the Search-R1 baselines.

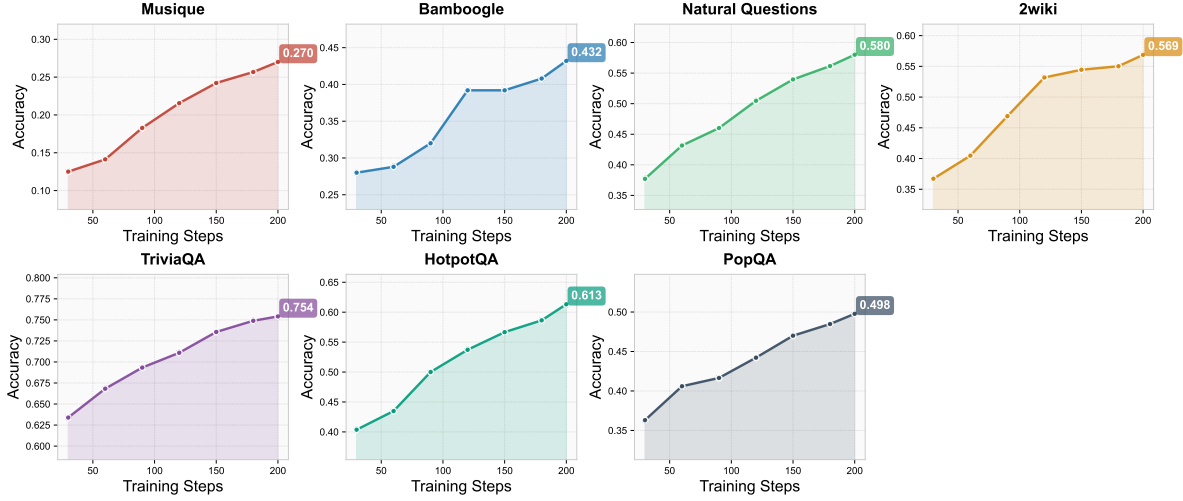


Figure 2: **Validation accuracy of DSPO across all seven QA benchmarks over the course of training.** Every curve trends steadily upward and none regresses, showing that DSPO’s training-reward gains translate directly into higher held-out accuracy rather than reward hacking. The tight coupling between rising reward and rising validation accuracy is direct evidence that the learned search-and-reasoning policy generalizes.

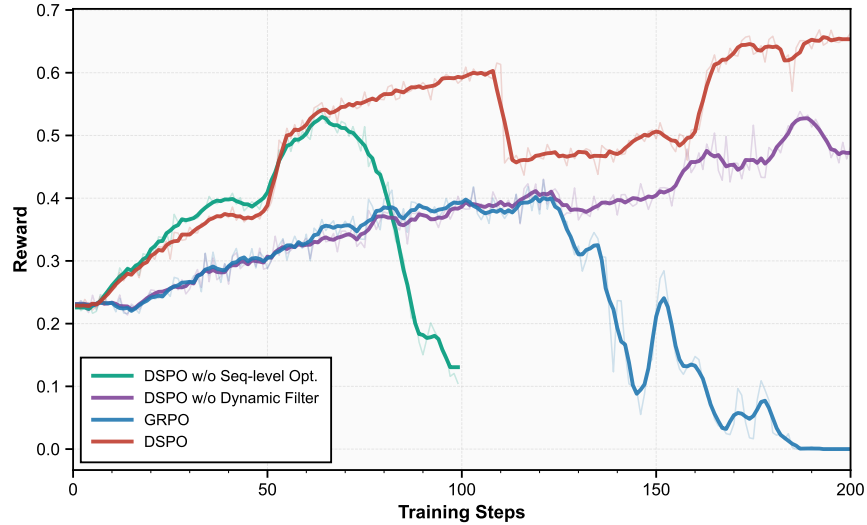


Figure 3: **Training-reward dynamics of DSPO versus its ablations and the GRPO baseline.** A side-by-side view of the learning curves makes the ordering unambiguous. DSPO (red) climbs stably and monotonically to the highest reward of any method. The two token-level variants (green, blue) briefly improve and then suffer catastrophic policy collapse, and the sequence-level variant without our dynamic filter (purple) stays stable but plateaus well below DSPO. The gap between DSPO and every alternative widens as training proceeds, quantifying the joint contribution of sequence-level optimization and dynamic filtering.

Benchmarks and Baselines. To make the comparison rigorous and reproducible, our experimental design follows the established protocol of Search-R1 (Jin et al., 2025b). We train on a composite dataset containing the training splits of Natural Questions (NQ) (Kwiatkowski et al., 2019) and HotpotQA (Yang et al., 2018), and we then measure generalization on the test sets of seven diverse QA benchmarks: NQ, TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2022), HotpotQA, 2WikiMultiHopQA (Ho et al., 2020), Musique (Trivedi et al., 2022), and Bamboogle (Press et al., 2022). Because only NQ and HotpotQA are seen in training, the remaining five benchmarks constitute an out-of-distribution generalization test.

Prompt Template. Answer the given question. You must conduct reasoning inside `<think>` and `</think>`. first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by `<tool_call>` query `</tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer>` Beijing `</answer>`. **Question:** ...

Table 1: The prompt template used in our experiments, reused verbatim from Search-R1 to keep the comparison controlled.

Our comparison suite is designed to separate algorithmic gains from confounds. On the external side we compare against the Qwen2.5-7B and 14B models trained with PPO and GRPO in the Search-R1 framework (Jin et al., 2025b;a), spanning both a same-sized (7B) and a larger (14B) reference point. On the internal side, two ablations isolate each DSPO component: (1) **DSPO w/o dynamic filter**, equivalent to GSPO (Zheng et al., 2025), and (2) **DSPO w/o sequence-level opt.**, which reverts to the strong token-level policy DAPO (Yu et al., 2025). Comparing DSPO to each of these four external and two internal points pins the contribution of every design choice.

For implementation, all training starts from the same Qwen2.5-7B-Instruct checkpoint. Our experiments are built on the VeRL framework (Sheng et al., 2025), adapting the provided Search-R1-like example code and scripts to our methodology. One point is essential for a fair comparison: because our work uses a modified reward function, we did not simply copy the published Search-R1 numbers—we retrained the PPO and GRPO baselines from the Search-R1 framework (Jin et al., 2025b;a) under our exact experimental conditions, and these retrained models serve as our primary external benchmarks. For transparency, we additionally report the original published EM scores in parentheses in Table 2 so the two reward regimes can be inspected side by side.

Implementation and Evaluation. To isolate the benefits of our algorithm, all RL experiments deliberately employ a single standard BM25 retriever, held fixed across every method. This controlled setup ensures that observed differences are attributable to the learned policy rather than to a stronger retriever, and it means the reported margins are, if anything, conservative. Across all methods, models are trained with the same sparse binary reward based on substring Exact Match (subEM) of the final answer, and subEM is likewise the single primary evaluation metric, so every number in our tables is measured on a common footing.

4.2 MAIN RESULTS AND ABLATION STUDY

Table 2 provides a single, holistic view of DSPO’s effectiveness by placing the main comparison and the ablation side by side. This layout lets the reader read the same DSPO column against four external Search-R1 baselines (7B/14B $\times$ GRPO/PPO) and its own two ablated variants at once, so both the size of the gain over prior work and the contribution of each component are visible in one glance. We now walk through each comparison in turn.

Comparison with Baselines. The results in Table 2 establish DSPO’s superiority along two clear axes. First, at equal model size: our DSPO-trained 7B agent reaches an average subEM of 0.531 versus 0.377 for the same-sized Search-R1 (GRPO, 7B) model, a 34.1% relative improvement, and it takes the best or second-best score on all seven benchmarks individually—not merely on the average. Second, across model size: this 7B agent edges out even the twice-as-large Search-R1 14B models on average (0.531 vs. 0.530 for the retrained GRPO-14B), and on the hardest multi-hop benchmarks its per-dataset margins are substantial, e.g., 0.613 vs. 0.563 on HotpotQA and 0.270 vs. 0.260 on Musique. That a 7B model can match a 14B model trained with the same framework is strong evidence that the gains come from a more effective and stable learning algorithm rather than from raw model scale.

Analysis of Ablations. The ablation columns in Table 2 let us attribute the full 0.531 average to its two sources, and both turn out to be indispensable. Removing the dynamic filter (‘w/o dynamic filter’) is the more damaging: the average collapses from 0.531 to 0.313, a loss of 0.218 absolute (about 41% of the full score), because the sequence-level objective is then starved of signal by homogeneous-reward batches. Removing sequence-level optimization (‘w/o sequence-level opt.’) drops the average to 0.406, a loss of 0.125 absolute; this token-level variant recovers more than the filter-less one but still trails full DSPO by 0.125 and, as the next section shows, is additionally prone to catastrophic training collapse. Read together, the two ablations bracket the full method from below (0.313 and 0.406 vs. 0.531) and confirm the components are complementary rather than redundant: sequence-level updates supply stability, while

Table 2: Comprehensive substring-EM comparison of DSPO against external baselines and internal ablations on seven QA benchmarks, all under an identical BM25 setup. External baselines are Search-R1 (7B & 14B) with GRPO and PPO (Jin et al., 2025b;a); ablations remove one component each (‘w/o dynamic filter’, ‘w/o seq-level opt.’). All models were retrained and evaluated under our adjusted reward to keep the comparison consistent, with the originally published Search-R1 EM scores shown in parentheses for reference. Reading the last row: our 7B DSPO reaches an average of **0.531**, exceeding the same-sized GRPO-7B baseline (0.377) by 34.1% relative and matching the much larger 14B GRPO model (0.530). Best results are in **bold**; second-best are underlined.

Dataset	Search-R1				DSPO & Ablations (Ours, 7B)		
	GRPO (7B)	PPO (7B)	GRPO (14B)	PPO (14B)	w/o dyn. filter	w/o seq-lvl opt.	DSPO
NQ	0.423 (0.429)	(0.393)	<u>0.535</u> (0.482)	(0.424)	0.363	0.470	0.580
TriviaQA	0.658 (0.623)	(0.610)	0.760 (0.667)	(0.660)	0.515	0.695	<u>0.754</u>
PopQA	0.395 (0.427)	(0.397)	<u>0.477</u> (0.434)	(0.442)	0.277	0.430	0.498
HotpotQA	0.401 (0.386)	(0.370)	<u>0.563</u> (0.429)	(0.436)	0.330	0.438	0.613
2WikiMultiHopQA	0.357 (0.414)	(0.346)	0.611 (0.424)	(0.379)	0.285	0.398	<u>0.569</u>
Musique	0.122 (0.162)	(0.146)	<u>0.260</u> (0.191)	(0.210)	0.105	0.133	0.270
Bamboogle	0.280 (0.400)	(0.368)	0.504 (0.492)	(0.480)	0.288	0.280	<u>0.432</u>
Average	0.377 (0.396)	(0.385)	<u>0.530</u> (0.446)	(0.433)	0.313	0.406	0.531

the dynamic filter converts sparse rewards into a dense, efficient learning signal, and only their combination attains the top score on every benchmark.

Beyond these aggregate metrics, DSPO also elicits qualitatively sophisticated search behaviors—recognizing irrelevant results, reformulating queries, and verifying answers over multiple turns (see Appendix A for detailed trajectory examples). Notably, all of these behaviors emerge from pure RL training under DSPO, without any supervised demonstrations, indicating that the quantitative gains are backed by genuinely improved search strategies.

4.3 ANALYSIS OF TRAINING DYNAMICS

To connect the end-of-training scores to the training process that produced them, we compare the reward trajectories of DSPO, its two ablations, and the GRPO baseline. Figure 3 makes the contrast stark and easy to rank.

DSPO (red) rises smoothly and monotonically, converging to the highest reward level of any curve in the figure—the training-time counterpart of its top 0.531 test average. The **token-level methods**, DSPO w/o Seq-level Opt. (green) and vanilla GRPO (blue), tell the opposite story: after a brief initial climb their rewards collapse outright, a direct manifestation of the high-variance, token-level gradient updates. **DSPO w/o Dynamic Filter** (purple) avoids collapse thanks to sequence-level updates but flattens out at a visibly lower ceiling, consistent with its reduced 0.313 test average. The single method that is simultaneously stable and high-reward is the full DSPO, which is precisely the regime where both components are active—the reward curves and the Table 2 scores tell one consistent story.

We further verify that this training-reward advantage is real generalization rather than reward hacking by tracking held-out validation accuracy throughout training. As shown in Figure 2, DSPO’s validation accuracy on NQ, HotpotQA, and the other benchmarks rises in lockstep with its reward curve and never regresses. The tight correlation between the two curves confirms that improvements in training reward are converted into genuine, generalizable search-and-reasoning ability.

4.4 SCALABILITY AND GENERALIZATION ANALYSIS

To test whether these gains persist beyond the 7B setting, we extend the evaluation along two axes: larger model scale and a different task domain.

Scalability to Larger Models. We ask whether DSPO’s advantage holds at 14B parameters by training Qwen2.5-14B-Instruct under the same protocol. Table 3 shows that it does, and by a wide margin. The DSPO-trained 14B model reaches an average subEM of 0.606, beating the strong GRPO-14B baseline (0.530) by 7.6 points absolute, a **14.3% relative** gain. The improvement is uniform: DSPO-14B is the best method on all seven benchmarks, with per-dataset

gains ranging from +5.5% on TriviaQA up to +38.8% on the hardest benchmark, Musique. This consistent, scale-preserving advantage indicates that DSPO converts additional model capacity into accuracy more effectively than the standard baseline.

Table 3: Scalability analysis on Qwen2.5-14B-Instruct, reported as substring-EM. The rightmost column gives DSPO-14B’s relative gain over the GRPO-14B baseline. DSPO-14B is best on every benchmark (bold) and lifts the average from 0.530 to **0.606** (+14.3% relative), with per-dataset gains from +5.5% (TriviaQA) to +38.8% (Musique).

Dataset	Instruct (14B)	GRPO (14B)	DSPO (7B)	DSPO (14B)	Gain
NQ	0.345	0.535	0.580	0.629	+17.6%
HotpotQA	0.407	0.563	0.613	0.665	+18.1%
2WikiMQA	0.332	0.611	0.569	0.699	+14.4%
Bamboogle	0.328	0.504	0.432	0.544	+7.9%
PopQA	0.364	0.477	0.498	0.545	+14.3%
TriviaQA	0.643	0.760	0.754	0.802	+5.5%
Musique	0.151	0.260	0.270	0.361	+38.8%
Average	0.367	0.530	0.531	0.606	+14.3%

Generalization to Mathematical Reasoning. Finally, we test whether DSPO’s benefits extend beyond search to single-turn mathematical reasoning, using the Qwen2.5 and Qwen3 model families. Table 4 reports the head-to-head comparison against GRPO on Math500 and Olympiad-Bench. DSPO wins both matchups: +2.6% relative on Math500 with Qwen2.5-Math-7B (0.798 vs. 0.772) and +2.7% relative on Olympiad-Bench with Qwen3-4B (0.755 vs. 0.728). Although these tasks require no retrieval, DSPO still comes out ahead at both model sizes, indicating that its stability and filtering principles transfer to general reasoning domains.

Table 4: Generalization to single-turn mathematical reasoning. DSPO outperforms GRPO in both settings (best in **bold**): +2.6% relative on Math500 and +2.7% relative on Olympiad-Bench, shown in the rightmost Gain column.

Model	Benchmark	Steps	GRPO	DSPO	Gain
Qwen2.5-Math-7B	Math500	200	0.772	0.798	+2.6%
Qwen3-4B	Olympiad-Bench	100	0.728	0.755	+2.7%

5 CONCLUSION

In this work, we tackled the critical instability and sample inefficiency that plague RL for autonomous LLM search agents. We introduced Dynamic-filter Sequence-level Policy Optimization (DSPO), an improved algorithm built on two components: sequence-level optimization to prevent catastrophic policy collapse, and a dynamic outcome-based filter to convert sparse rewards into a consistently effective learning signal. The empirical case is strong and consistent. Under a matched BM25 setup, DSPO raises the seven-benchmark average to 0.531—a 34.1% relative gain over the same-sized GRPO baseline—while a 7B DSPO model matches the twice-as-large 14B Search-R1 model and surpasses it on the hardest multi-hop benchmarks (0.613 vs. 0.563 on HotpotQA). The advantage persists at 14B scale (0.606 vs. 0.530, +14.3% relative) and even transfers to single-turn math reasoning, and throughout, DSPO trains along a smooth, non-collapsing reward curve where token-level baselines fail.

By enabling robust training from environmental feedback alone, DSPO establishes a practical and efficient blueprint for building capable LLM agents without costly expert data—and it does so using only a basic BM25 retriever, so the reported margins understate what a stronger retrieval pipeline could add. With this stable foundation, future work can confidently explore integrating advanced retrievers or extending DSPO to complex, multi-tool tasks. Because sparse rewards and unstable policy gradients are not unique to search, and because our early math-reasoning results already show consistent (if modest) gains, we expect DSPO’s principles to yield similar performance and stability improvements in domains such as mathematics and code generation—a promising direction for broader validation. We believe the core tenets of DSPO—matching the optimization unit to the reward signal and guaranteeing signal density—will be instrumental in developing the next generation of autonomous AI.

REFERENCES

- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- Tuhin Chakrabarty, Vishakh Padmakumar, Faeze Brahman, and Smaranda Muresan. Creativity support in the age of large language models: An empirical study involving professional writers. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 132–155, 2024.
- Mingyang Chen, Tianpeng Li, Haoze Sun, Yijie Zhou, Chenzheng Zhu, Haofen Wang, Jeff Z Pan, Wen Zhang, Huajun Chen, Fan Yang, et al. Learning to reason with search for llms via reinforcement learning. *arXiv preprint arXiv:2503.19470*, 2025.
- Paul F Christiano, Jan Leike, Tom Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences. *Advances in neural information processing systems*, 30, 2017.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, et al. The entropy mechanism of reinforcement learning for reasoning language models. *arXiv preprint arXiv:2505.22617*, 2025.
- Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, Haofen Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2(1), 2023.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Sercan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Sercan Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-rl: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. *arXiv preprint arXiv:1705.03551*, 2017.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP (1)*, pp. 6769–6781, 2020.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:453–466, 2019.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding rl-zero-like training: A critical perspective. *arXiv preprint arXiv:2503.20783*, 2025.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Hannaneh Hajishirzi, and Daniel Khashabi. When not to trust language models: Investigating effectiveness and limitations of parametric and non-parametric memories. *arXiv preprint arXiv:2212.10511*, 7, 2022.
- Guillermo Marco, Julio Gonzalo, Ramón del Castillo, and María Teresa Mateo Girona. Pron vs prompt: Can large language models already challenge a world-class fiction author at creative text writing? *arXiv preprint arXiv:2407.01119*, 2024.

- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. Measuring and narrowing the compositionality gap in language models. *arXiv preprint arXiv:2210.03350*, 2022.
- Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. *Advances in neural information processing systems*, 36:53728–53741, 2023.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551, 2023.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Guangming Sheng, Chi Zhang, Zilinfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. In *Proceedings of the Twentieth European Conference on Computer Systems*, pp. 1279–1297, 2025.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.
- Trieu H Trinh, Yuhuai Wu, Quoc V Le, He He, and Thang Luong. Solving olympiad geometry without human demonstrations. *Nature*, 625(7995):476–482, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022.
- Junde Wu, Jiayuan Zhu, Yuyuan Liu, Min Xu, and Yueming Jin. Agentic reasoning: A streamlined framework for enhancing llm reasoning with agentic tools. *arXiv preprint arXiv:2502.04644*, 2025.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversations. In *First Conference on Language Modeling*, 2024.
- John Yang, Carlos E Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. *Advances in Neural Information Processing Systems*, 37:50528–50652, 2024.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.

-
- Wayne Xin Zhao, Kun Zhou, Junyi Li, Tianyi Tang, Xiaolei Wang, Yupeng Hou, Yingqian Min, Beichen Zhang, Junjie Zhang, Zican Dong, et al. A survey of large language models. *arXiv preprint arXiv:2303.18223*, 1(2), 2023.
- Yuzhong Zhao, Yue Liu, Junpeng Liu, Jingye Chen, Xun Wu, Yaru Hao, Tengchao Lv, Shaohan Huang, Lei Cui, Qixiang Ye, et al. Geometric-mean policy optimization. *arXiv preprint arXiv:2507.20673*, 2025.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.
- Qinkai Zheng, Xiao Xia, Xu Zou, Yuxiao Dong, Shan Wang, Yufei Xue, Lei Shen, Zihan Wang, Andi Wang, Yang Li, et al. Codegeex: A pre-trained model for code generation with multilingual benchmarking on humaneval-x. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pp. 5673–5684, 2023.

A APPENDIX

To show what underlies the quantitative gains reported in the main results, Table A.1 presents representative trajectories from our model. Each example illustrates one concrete search skill that DSPO learns from reward alone: (1) reformulating queries when an initial search returns irrelevant results, (2) verifying information through follow-up searches, and (3) extracting complete answers directly from retrieved evidence. These behaviors are the mechanism behind the higher subEM scores, particularly on the multi-hop benchmarks where iterative search matters most.

Table A.1: Representative trajectories that illustrate the search-and-reasoning skills behind DSPO’s quantitative gains. Across the three examples the model exhibits **(a)** recognition of irrelevant results, **(b)** query reformulation, and **(c)** verification strategies—each ending in a correct answer.

Example 1: Presidential Succession (<i>Query Refinement after Failure</i>)	
Question	Who is next in line if something happens to the president?
Gold	vice president
Turn 1	<code>search("who is next in line if something happens to the president")</code> → Retrieved: Documents about novels "Something Happened" and "Mosquitoes Buzz" Agent: "The queries were not directly about the line of succession... The relevant information I need is not present in the search results. Let's try searching again with a more specific query."
Turn 2	<code>search("line of succession for the president")</code> → Retrieved: Philippine and US presidential succession information Agent: "Based on the search results... the next person in line would be the Vice President."
Answer	✓ The Vice President
Example 2: University Identification (<i>Information Verification</i>)	
Question	Which American University is situated in Ithaca, New York?
Gold	Cornell University
Turn 1	<code>search(["American University in Ithaca", "University in Ithaca NY"])</code> → Retrieved: "Ithaca is home to Cornell University, an Ivy League school..." Agent: "Ithaca, New York is home to Cornell University, which is an Ivy League school. However... it is highly likely that the university in question is Cornell. Let me confirm this."
Turn 2	<code>search(["Cornell University location", "Cornell University New York"])</code> → Retrieved: Cornell history confirming establishment in Ithaca Agent: "Cornell University is indeed located in Ithaca, New York."
Answer	✓ Cornell University
Example 3: Biographical Information (<i>Direct Extraction</i>)	
Question	What is John Andrew Martin’s occupation?
Gold	lawyer, attorney, politician
Turn 1	<code>search(["John Andrew Martin occupation", "John Andrew Martin job"])</code> → Retrieved: "American journalist, attorney, soldier, and politician..." Agent: "John Andrew Martin was an American <u>journalist, attorney, soldier, and politician</u>. He represented Colorado in the U.S. House of Representatives."
Answer	✓ Journalist, attorney, soldier, and politician